Department of ECE

19ECE304 Microcontrollers and Interfacing Lab

Project Report- December 2024

Project Title: SMART MEDICINE REMINDER WITH HEARTBEAT DETECTION AND RTC ALARM SYSTEM

Team Members (Name and Roll No)

AM.EN.U4ECE22024 – M.Y.RAVI TEJA
AM.EN.U4ECE22028 – N.VIVEKANANDA
AM.EN.U4ECE22035 – P.JHANSI LAKSHMI
AM.EN.U4ECE22055 – K.SASANK
AM.EN.U4ECE22058 – S.SAI SRI SAHITHI

Signature of faculty with Date:

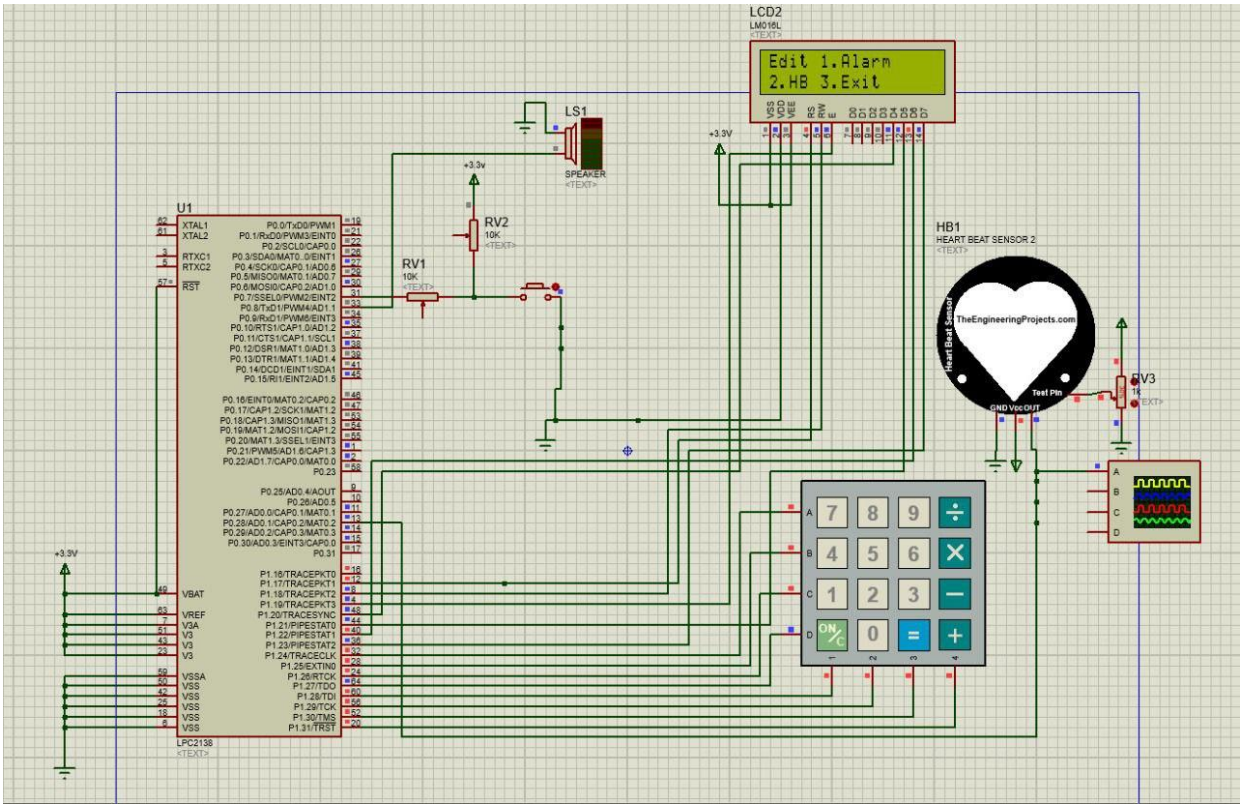
Project Title: SMART MEDICINE REMINDER WITH HEARTBEAT DETECTION AND RTC ALARM SYSTEM

Objective: The objective of a Smart Medicine Reminder With Heartbeat Detection And Rtc Alarm System using Proteus software is to design and simulate a system that The aim of this project is to design and implement a Smart Medicine Reminder system that integrates heartbeat detection and an RTC alarm. The system provides user-friendly options for monitoring heartbeat, setting medication reminders, and ensuring timely medication intake.

Components Required:

LPC2148,
LCD,
4X4 MATRIX KEYPAD,
SPEAKER,
HEART BEAT SENSOR,
PUSH BUTTON

Block Diagram/Circuit Diagram:



THEORY:

(1) LPC2148

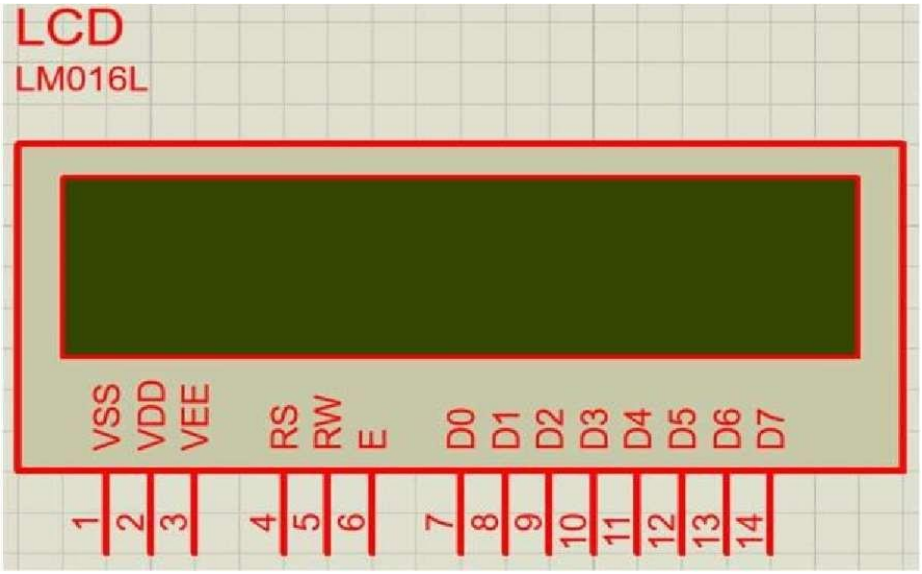
LPC2148 is ARM7 based microcontroller. LPC2138 is an ARM7TDMI-S based high performance 32-bit RISC Microcontroller with Thumb extensions 512KB on-chip Flash ROM with In-System Programming (ISP) and In-Application Programming (IAP), Two 8-ch 10bit ADC 32KB RAM, Vectored Interrupt Controller, Two UARTs, one with full modem interface. Two I2C serial interfaces, Two SPI serial interfaces Three 32-bittimers, Watchdog Timer, Real Time Clock with optional battery backup, Brown out detect circuit General purpose I/O pins. CPU clock up to 60 MHz, On-chip crystal oscillator and On-chip PLL.

(2) LCD

An LCD (Liquid Crystal Display) screen is electronic display module and has a wide range of applications. A 16x2 LCD display is very basic module and is very commonly used in various devices and circuits. A 16x2 LCD means it can display 16 characters per line and there are 2 such lines. In this LCD each character is displayed in 5x7 pixel matrix. The 16 x 2 intelligent alphanumeric dot matrix display is capable of displaying 224 different characters and symbols. This LCD has two registers, namely, Command and Data.

Command register stores various commands given to the display. Data register stores data to be displayed. The process of controlling the display involves putting the data that form the image of what you want to display into the data registers, then putting instructions in the instruction register.

This whole process is managed by the LPC2148 microcontroller



3. Keypad (4x4):
The keypad provides a user interface to select options like setting alarms or initiating heartbeat monitoring. It communicates with the microcontroller through GPIO pins using a row-column scanning mechanism.
4. Heartbeat Sensor (HB1):
The heartbeat sensor detects pulse rates by converting the analog signal into a voltage that is fed to the ADC pin of the microcontroller. It allows real-time monitoring of heart rate data.
5. Speaker (LS1):
The buzzer generates audio alerts for reminders or system notifications, such as when an alarm is triggered. It is controlled by a GPIO or PWM pin of the microcontroller.
6. Resistors (RV1, RV2):
Potentiometers like RV1 and RV2 are used to adjust the contrast of the LCD and fine-tune input signals to the microcontroller, ensuring accurate readings and display clarity.
7. Oscillator (XTAL1, XTAL2):
The crystal oscillator provides a stable clock signal to the microcontroller, ensuring precise timekeeping for the RTC and proper operation of the system.
8. Power Supply (3.3V):
A regulated 3.3V power source supplies voltage to the microcontroller, LCD, and other peripherals, ensuring stable operation of the circuit.

Description of Peripherals Used:
ADC-:
The ADC in the LPC2148 microcontroller converts analog signals into digital values, enabling the microcontroller to process data from sensors and other analog devices. It features multiple channels (AD0.0 to AD0.7 and AD1.0 to AD1.7), each capable of accepting analog inputs. The ADC operates with a resolution of 10 bits, providing precise conversion suitable for applications like temperature sensing, heartbeat monitoring, and voltage measurement. This peripheral is critical for interfacing the analog world with digital systems.

ADC- ANALOG TO DIGITAL CONVERTOR

- LPC2148 has two inbuilt 10-bit Successive Approximation ADC. ADC0 has six channels (AD0.1-AD0.6). ADC1 has 8-Channels (AD1.0-AD1.7). ADC operating frequency is 4.5 MHz (max.), operating frequency decides the conversion time.
- The ADC reference voltage is measured across GND to VREF, meaning it can do the conversion within this range. Usually, the VREFPis connected to VDD.

Features:-

- 10-bit successive approximation analog to digital converter (one in LPC2141/2 and two in LPC2144/6/8).
- Input multiplexing among 6 or 8 pins (ADC0 and ADC1).
- Power-down mode.
- Measurement range 0 V to VREF (typically 3 V; not to exceed VDDA voltage level).
- 10 bit conversion time ≥2.44 µs.
- Burst conversion mode for single or multiple inputs.
- Optional conversion on transition on input pin or Timer Match signal.
- Global Start command for both converters (LPC2144/6/8 only).

1. Sensor Interfacing:

- The ADC in the LPC2148 microcontroller facilitates the integration of analog sensors by converting their outputs into digital values for processing. For example, a heartbeat sensor generates an analog voltage proportional to the pulse rate.
- The ADC reads this signal and provides digital data to the microcontroller, enabling real-time monitoring and analysis. This capability allows the system to interact seamlessly with the physical world through various environmental and biomedical sensors

2. Voltage Monitoring:

- The ADC also plays a crucial role in monitoring voltage levels within the system. For instance, it can measure the power supply voltage or battery levels to ensure system stability and functionality.
- By connecting a potentiometer or voltage divider to the ADC, the microcontroller can dynamically adjust or calibrate system parameters based on real-time voltage readings.

3. Precision Measurements:

- With its 10-bit resolution, the ADC provides precise digital representations of analog signals, making it ideal for applications requiring high accuracy.
- This precision is particularly useful in scenarios like biomedical signal processing, where small voltage variations need to be captured accurately, or in instrumentation systems for precise environmental monitoring and control

SUMMARY OF PERIPHERAL USAGE:

Analog-to-Digital Converter (ADC):

The ADC in the LPC2148 microcontroller converts analog signals into digital values, enabling the microcontroller to process data from sensors and other analog devices. It features multiple channels (AD0.0 to AD0.7 and AD1.0 to AD1.7), each capable of accepting analog inputs. The ADC operates with a resolution of 10 bits, providing precise conversion suitable for applications like temperature sensing, heartbeat monitoring, and voltage measurement. This peripheral is critical for interfacing the analog world with digital systems.

Real-Time Clock (RTC):

The RTC is a dedicated peripheral for accurate timekeeping and calendar functionalities. It operates independently using a battery backup connected to the VBAT pin, ensuring uninterrupted operation even during power outages. The RTC module includes time (hour, minute, second) and date (day, month, year) tracking, making it essential for applications requiring scheduled events or alarms, such as medicine reminders. An external crystal oscillator enhances its accuracy.

General Purpose Input/Output (GPIO):

GPIO pins provide flexible digital input and output capabilities, allowing the microcontroller to interact with various peripherals. In LPC2148, GPIO pins are organized into two ports (P0 and P1), offering a wide range of functionalities like interfacing with LEDs, keypads, sensors, and displays. Each pin can be individually configured for input or output and supports multiplexed functionality, making GPIO a versatile and essential interface for embedded system

Project Description / Working:

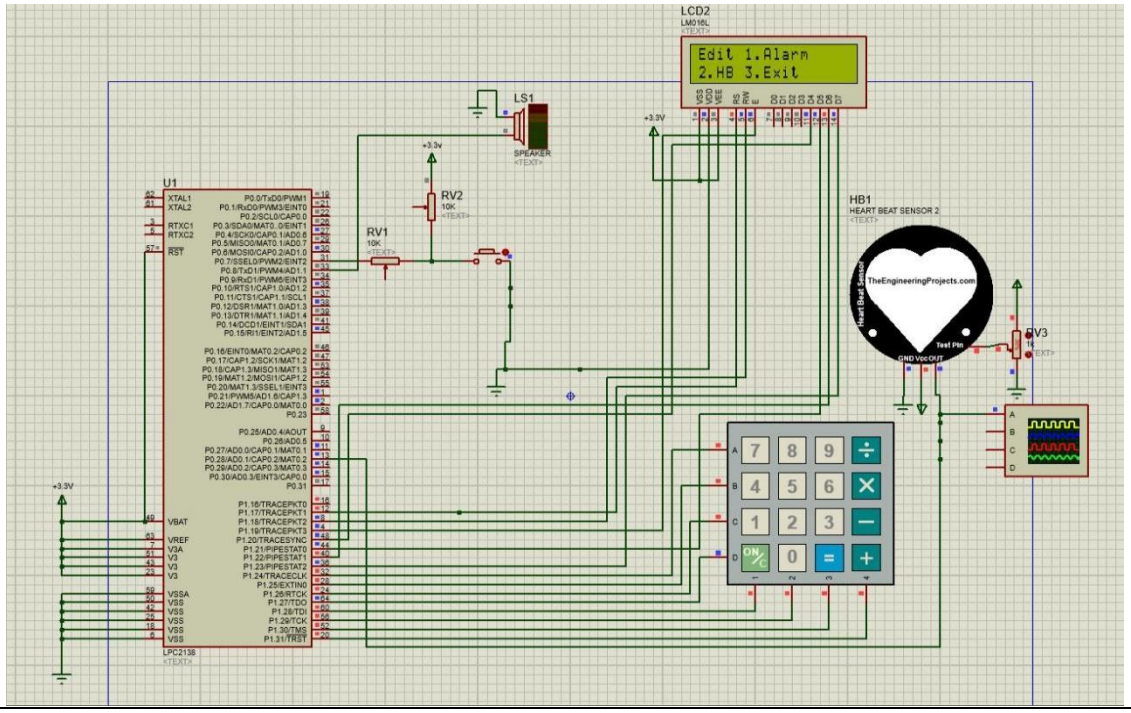
This project implements a microcontroller-based system with a real-time clock and alarm system integrated with a heartbeat monitoring feature. The system works as follows:

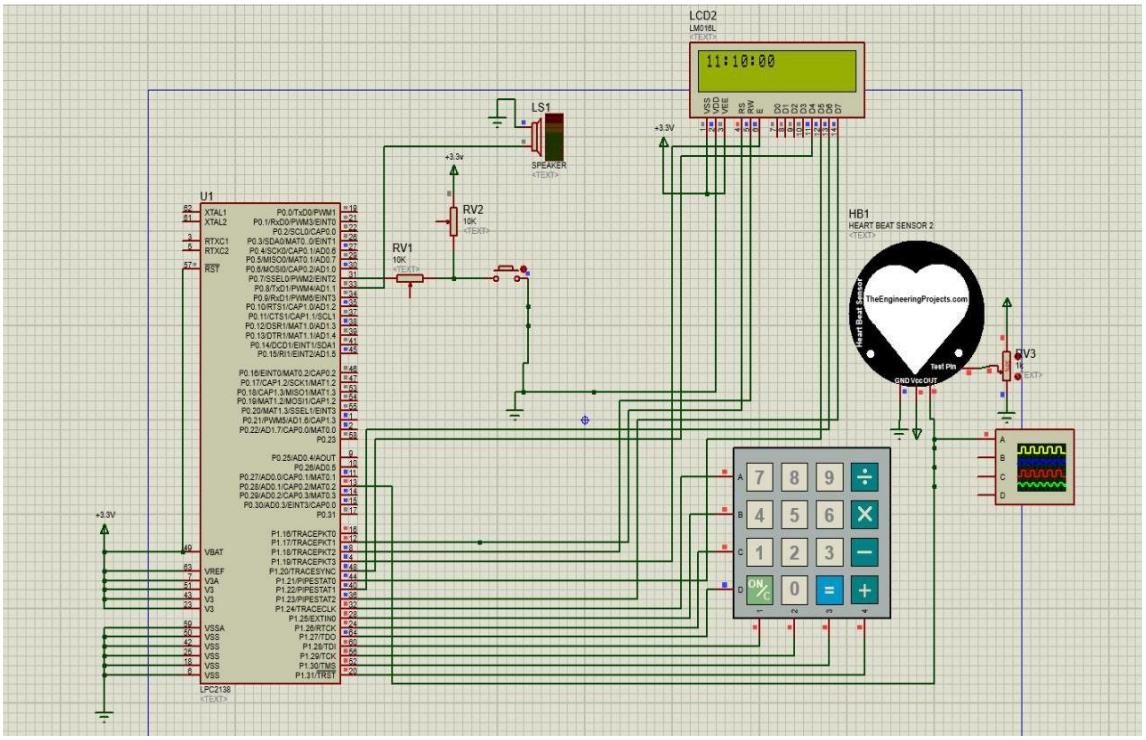
1. **User Input via Keypad:**
   The user interacts with the system through the keypad to set alarms or access the heartbeat monitoring function.
2. **Alarm Management:**
   Alarms are set through the keypad, and when the scheduled time is reached, the buzzer is activated to notify the user. The medication reminder is displayed on the LCD.
3. **Heartbeat Monitoring:**
   The system uses a heartbeat sensor to detect pulse rates. The analog signal from the potentiometer sensor is processed via the ADC of the LPC2138 microcontroller, and the results are displayed on the LCD. Additionally, the digital oscilloscope visualizes the heartbeat waveform in real-time.
4. **LCD Feedback:**
   The LCD provides real-time updates, showing current time, alarms, and heartbeat data for user convenience.

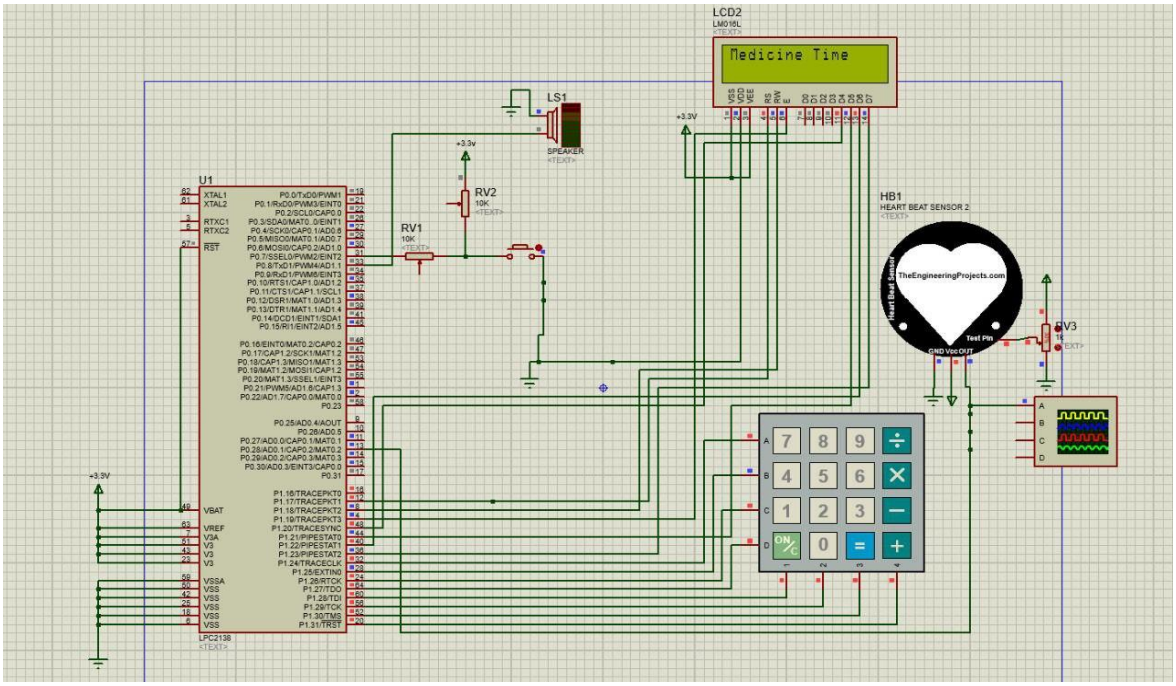
RESULTS-:

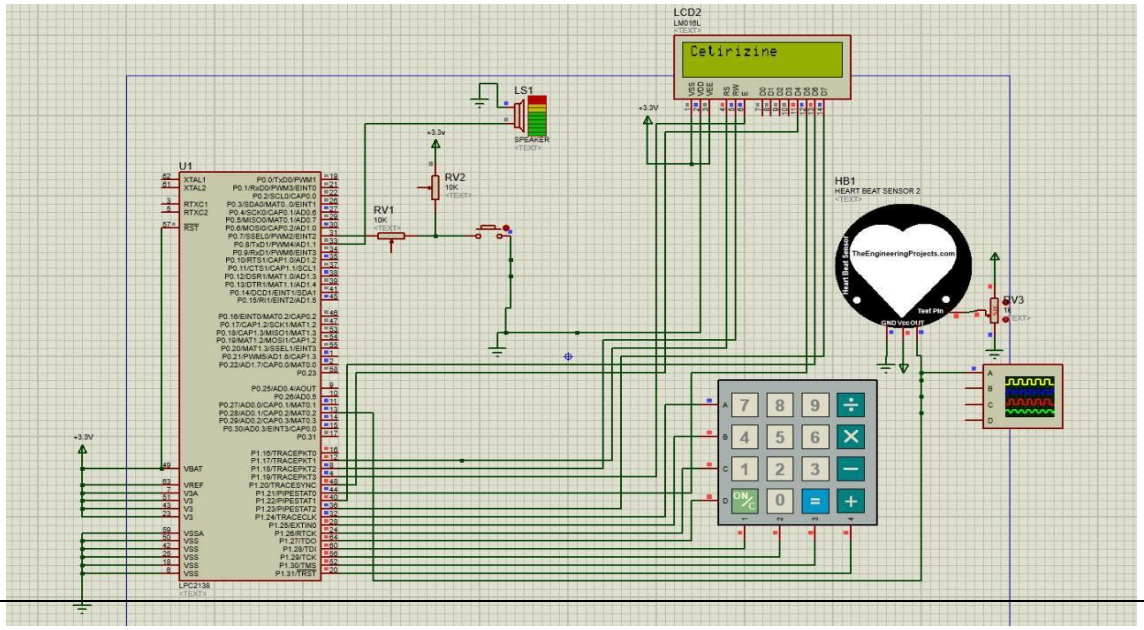
- **Accurate Timekeeping:** The RTC module successfully maintains precise time.
- **Efficient Alarm Notifications:** Medication reminders are effectively managed and delivered with audible and visual alerts.
- **Real-Time Health Monitoring:** The heartbeat sensor detects pulse rates accurately, with outputs displayed on both the LCD and digital oscilloscope.

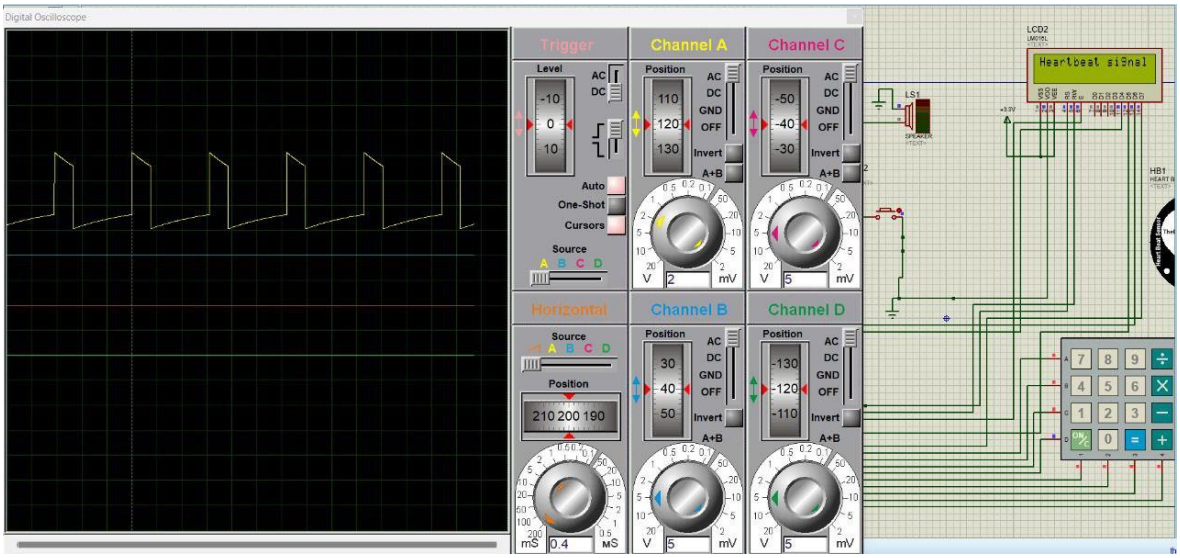
SIMULATION RESULTS :




LCD2
LM016L
11:10:00
LS1
SPEAKER
RV1
10K
RV2
10K
U1
LPC2138
HB1
HEART BEAT SENSOR 2
TheEngineeringProjects.com
RV3
+3.3V


LCD2
LM016L
Medicine Time
LS1
SPEAKER
RV1
10K
RV2
10K
U1
LPC2138
HB1
HEART BEAT SENSOR 2
TheEngineeringProjects.com
RV3
+3.3V


LCD2
LM016L
Cetirizine
LS1
SPEAKER
RV1
10K
RV2
10K
U1
LPC2138
HB1
HEART BEAT SENSOR 2
TheEngineeringProjects.com
RV3
+3.3V



Conclusion: The project successfully integrates an RTC (Real-Time Clock) with an alarm system and a user-interactive keypad, demonstrating a functional and efficient design for a medicine reminder system. The RTC ensures precise timekeeping, while the alarm feature alerts users at pre-set times with both visual and auditory cues. The LCD provides real-time feedback and clear instructions, enhancing usability. The interrupt-driven architecture ensures responsiveness and efficient use of the microcontroller's resources. This system has potential applications in healthcare and home automation, highlighting its practical relevance and effectiveness in addressing time-based reminders for medication adherence.

References:

https://github.com/mandar6988/keypad

The above project serves as a reference to our project, providing a foundational concept for implementing a **Smart Medicine Reminder With Heartbeat Detection And Rtc Alarm System** using the **LPC2148** microcontroller with the help of Analog to Digital Converter(ADC) peripheral.

Acknowledgement: I would like to thank our mentor Nagaraj sir , Chinmayi Mam Nisha mam for giving us this excellent opportunity to do a project on microcontrollers which helped to enhance our knowledge on software’s like Proteus and ARM-Keil, without their continuous support this project wouldn’t have been possible. I would also like to thank my teammates for their efforts throughout the project and making it a successful

Appendix: Code with Comments :

```c
#include<lpc213x.h>                 // Includes header file for LPC213x microcontroller
#include<stdio.h>                   // Standard Input/Output library for functions like sprintf

#include "LCD Header.h"             // Header file for LCD-related functions
#include "edit header.h"            // Header file for edit function handling

#define fpclk 15000000              // Defines the peripheral clock frequency
#define MASKSEC 0x3F                // Mask to extract seconds from RTC register
#define MASKMIN 0x3F00              // Mask to extract minutes from RTC register
#define MASKHOUR 0xF0000            // Mask to extract hours from RTC register
#define BUZZ 8                      // Defines GPIO pin for buzzer control (Port0.8)

void delay(void);                   // Function prototype for delay function

// KEYPAD Interrupt Service Routine
void ISR_KEYPAD(void) __irq
{
    edit_function();                // Call the edit function for handling keypad actions
    EXTINT = 0x04;                  // Clear external interrupt flag for keypad
    VICVectAddr = 0x00;             // Acknowledge the interrupt to the vector controller
}

// RTC Interrupt Service Routine
void ISR_RTC(void) __irq
{
    char str[20];                   // Buffer to store formatted time string
    static int alarm_triggered = 0; // Flag to track if the alarm has already triggered

    if (ILR & 0x01)                 // Check if RTC Counter Increment Interrupt occurred
    {
        sec = CTIME0 & MASKSEC;  // Extract seconds from RTC register
        min = (CTIME0 & MASKMIN) >> 8; // Extract minutes
        hour = (CTIME0 & MASKHOUR) >> 16; // Extract hours
```

```c
        ILR = 0x01;                 // Clear RTC counter interrupt flag
        LCD_CMD(0x85);              // Move LCD cursor to display time
        sprintf(str, "%02d:%02d:%02d", hour, min, sec); // Format time string
        LCD_INIT();                 // Reinitialize LCD
        LCD_STRING(str);            // Display the time on the LCD
    }

    if (ILR & 0x02) {               // Check if RTC Alarm Interrupt occurred
        if (hour == ALHOUR && min == ALMIN) { // Check if current time matches alarm time
            if (!alarm_triggered) {          // If alarm not already triggered
                LCD_INIT();                  // Reinitialize LCD
                LCD_STRING("Medicine Time"); // Display "Medicine Time" message
                delay();                     // Short delay
                LCD_INIT();                  // Clear LCD
                LCD_CMD(0x80);               // Move cursor to first line
                LCD_STRING("Paracetamol");   // Display first medicine name
                delay();                     // Short delay
                LCD_INIT();                  // Clear LCD
                LCD_CMD(0x80);               // Move cursor to first line
                LCD_STRING("Cetirizine");    // Display second medicine name
                alarm_triggered = 1;         // Set alarm triggered flag
            }
            IOSET0 = 1 << BUZZ;              // Turn on buzzer
            delay();                         // Delay for 1 second
            delay();                         // Delay for 2 seconds
            delay();                         // Delay for 3 seconds
            delay();                         // Delay for 4 seconds
            delay();                         // Delay for 5 seconds
            delay();                         // Delay for 6 seconds
            IOCLR0 = 1 << BUZZ;              // Turn off buzzer
        } else {
            alarm_triggered = 0;             // Reset alarm trigger flag if time doesn't match
        }
```

```
67
68          ILR = 0x02;                             // Clear alarm interrupt flag
69      }
70
71      VICVectAddr = 0;                            // Acknowledge interrupt to vector controller
72  }
73
74  // RTC INITIALIZATION
75  void RTC_Init()
76  {
77      ILR = 0x01;                                 // Enable RTC counter interrupt
78      CCR = 0x02;                                 // Enable calibration mode for RTC
79      CIIR = 0x07;                                // Enable increment interrupt for seconds, minutes, and hours
80      PREINT = (int)(fpclk / 32768) - 1;          // Set integer part of prescaler
81      PREFRAC = (int)(fpclk - ((PREINT + 1) * 32768)); // Set fractional part of prescaler
82      SEC = 5;                                    // Set initial seconds value
83      MIN = 9;                                    // Set initial minutes value
84      HOUR = 11;                                  // Set initial hours value
85      ALSEC = 4;                                  // Set alarm seconds value
86      ALMIN = 9;                                  // Set alarm minutes value
87      ALHOUR = 11;                                // Set alarm hours value
88      CCR = 0x01;                                 // Enable RTC
89      VICIntSelect &= (~(1 << 13));               // Configure interrupt as IRQ
90      VICVectCntl1 = 0x20 | 13;                   // Assign RTC interrupt to vector slot 1
91      VICVectAddr1 = (long)ISR_RTC;               // Set ISR address for RTC interrupt
92      VICIntEnable |= (1 << 13);                  // Enable RTC interrupt
93  }
94
95  // KEYPAD INTERRUPT INITIALIZATION
96  void keypad_interrupt_init()
97  {
98      PINSEL0 &= (~((1 << 15) | (1 << 14)));      // Configure P0.7 as GPIO
99      PINSEL0 |= (0x3) << 14;                     // Enable interrupt functionality on P0.7
```

```
94
95  // KEYPAD INTERRUPT INITIALIZATION
96  void keypad_interrupt_init()
97  {
98      PINSEL0 &= (~((1 << 15) | (1 << 14)));      // Configure P0.7 as GPIO
99      PINSEL0 |= (0x3) << 14;                     // Enable interrupt functionality on P0.7
100     EXTINT = 0x04;                              // Clear external interrupt flag
101     EXTMODE = 0x04;                             // Configure interrupt as edge-sensitive
102     EXTPOLAR = 0x00;                            // Configure interrupt as falling-edge
103     VICIntSelect &= (~(1 << 16));               // Configure interrupt as IRQ
104     VICVectCntl0 = 0x20 | 16;                   // Assign keypad interrupt to vector slot 0
105     VICVectAddr0 = (long)ISR_KEYPAD;            // Set ISR address for keypad interrupt
106     VICIntEnable |= (1 << 16);                  // Enable keypad interrupt
107 }
108
109 int main()
110 {
111     PINSEL2 = 0x00000000;                       // Set Port2 as GPIO
112     PINSEL0 = 0x00;                             // Set Port0 as GPIO
113     IODIR0 = 1 << BUZZ;                         // Set Port0.8 as output (buzzer pin)
114     LCD_INIT();                                 // Initialize LCD
115     LCD_CMD(0x80);                              // Move LCD cursor to first line
116     LCD_STRING("Time:");                        // Display "Time:" on LCD
117     LCD_CMD(0xC0);                              // Move cursor to second line
118     LCD_STRING("Press EDIT");                   // Display "Press EDIT" on LCD
119     RTC_Init();                                 // Initialize RTC
120     keypad_interrupt_init();                    // Initialize keypad interrupt
121     while (1)                                   // Infinite loop
122     {
123         // Main loop remains empty as system relies on interrupts
124     }
125     return 0;                                   // Return 0 (not reached in this code)
126 }
```

```
126 }
127
128 // Delay Function (Optimized)
129 void delay()
130 {
131     unsigned int i, j;
132     for (i = 0; i < 1100; i++) {                // Outer loop for delay duration
133         for (j = 0; j < 1000; j++);             // Inner loop to create delay
134     }
135 }
```